PROJECT: Scalable Microservices Architecture with ECS and Fargate

Services: ECS, Fargate, ECR, RDS, CloudWatch
Description: Containerize a microservices-based application using Docker. Deploy using ECS Fargate and manage logs with CloudWatch. Use RDS as the backend database.
Skills: Containers, microservices, monitoring

Definition: A Scalable Microservices Architecture with ECS and Fargate is a cloud-based system design approach where an application is broken into small, independent services (microservices) that can be developed, deployed, and scaled separately. These services are typically containerized using Docker and managed using Amazon Elastic Container Service (ECS), while AWS Fargate is used to run those containers without managing servers (serverless containers). Container images are stored in Amazon Elastic Container Registry (ECR), and data is often managed using Amazon RDS

Scope of the project:

The scope of this project focuses on building and deploying a scalable microservice architecture in the cloud using AWS services. It covers containerization of applications, storing images in ECR, and deploying containers using ECS Fargate. The project also includes monitoring application logs through CloudWatch for troubleshooting and performance monitoring. This approach can be extended to deploy multiple microservices and integrate additional services such as databases, load balancers, and CI/CD pipelines.

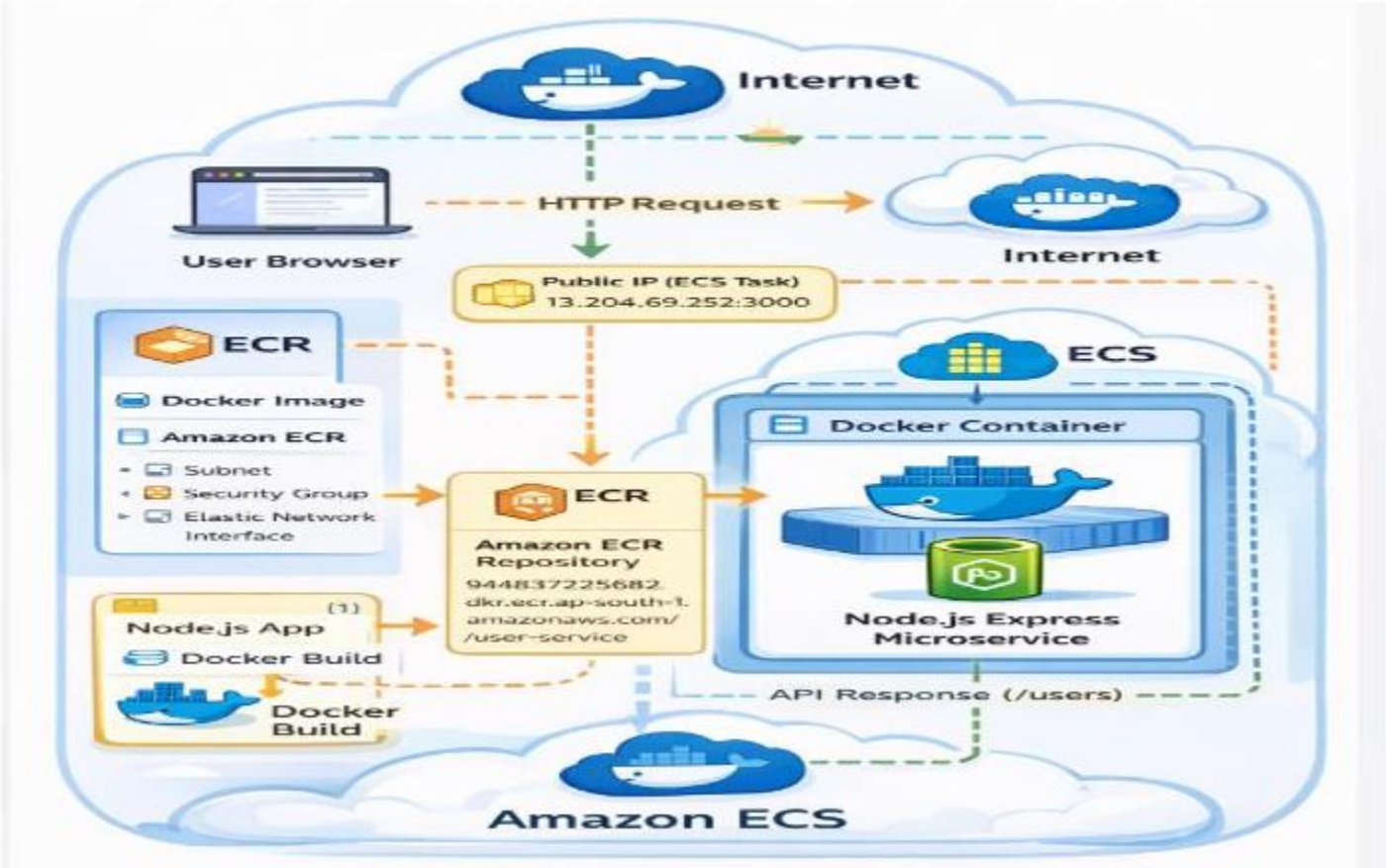
Methodologies:

1. Design the application as independent microservices (each service handles one function).
2. Containerize each microservice using Docker.
3. Push Docker images to Amazon Elastic Container Registry (ECR).
4. Create task definitions for each service in Amazon Elastic Container Service (ECS).
5. Deploy containers using AWS Fargate (serverless compute).
6. Configure networking using VPC, subnets, and security groups.
7. Connect microservices to database using Amazon RDS.
8. Set up load balancing using Application Load Balancer (ALB).
9. Enable logging and monitoring with Amazon CloudWatch.
10. Implement auto scaling based on CPU/memory usage.
11. Continuously integrate and deploy using CI/CD pipeline (optional Jenkins/GitHub Actions).
12. Test each microservice independently and as a full system.

Services Used in this Project:

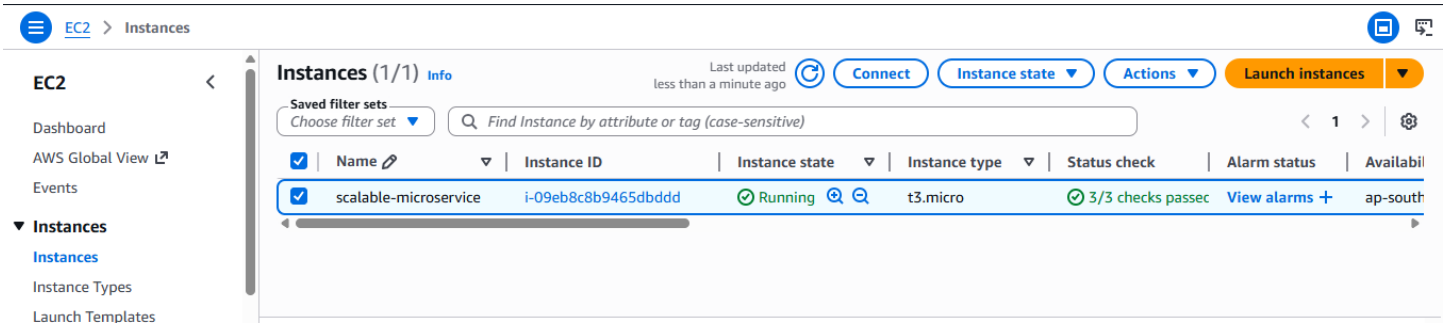
1. Amazon Elastic Container Service (ECS) – to deploy and manage microservices containers
2. AWS Fargate – to run containers without managing servers
3. Amazon Elastic Container Registry (ECR) – to store Docker images
4. Amazon RDS – to manage relational database backend
5. Amazon CloudWatch – to monitor logs and metrics

Architecture Diagram:

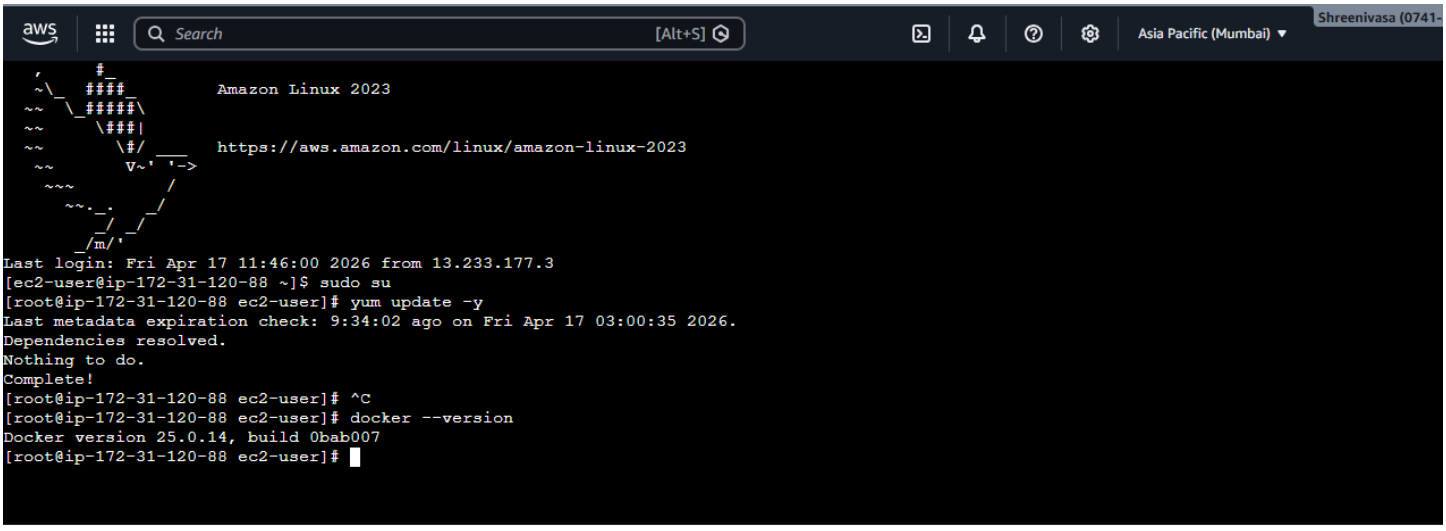


Implementation and Execution of Scalable Microservices Architecture with ECS and Fargate.

STEP 1: Launch an instance



STEP 2: Update the Server and Install Docker and verified Docker



STEP 3: Create a directory name Microservices Project

STEP 4: Create application file

```
[root@ip-172-31-120-88 microservices-project]# cd user-service/
[root@ip-172-31-120-88 user-service]# nano app.js
[root@ip-172-31-120-88 user-service]#
```

```
  GNU nano 8.3                                  app.js
const express = require('express');
const app = express();

app.get('/', (req, res) => {
    res.send('Hello from Microservice!');
});

app.listen(3000, '0.0.0.0', () => {
    console.log('Server running on port 3000');
});
```

STEP 6: Install Dependencies

```
[ec2-user@ip-10-0-2-213 user-service]$ npm init -y
Wrote to /home/ec2-user/microservices-project/user-service/package.json:

{
  "name": "user-service",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "description": ""
}


[ec2-user@ip-10-0-2-213 user-service]$
```

```
[ec2-user@ip-10-0-2-213 user-service]$ npm install express

added 65 packages, and audited 66 packages in 3s

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
[ec2-user@ip-10-0-2-213 user-service]$
```

STEP 7: Create Dockerfile

```
[root@ip-172-31-120-88 user-service]# nano Dockerfile
[root@ip-172-31-120-88 user-service]#
```

```
  GNU nano 8.3                                                    Dockerfile
FROM node:18

WORKDIR /app

COPY . /app

RUN npm install

CMD ["node", "./app.js"]
```

STEP 8: Build Docker Image

```
[root@ip-172-31-120-88 user-service]# docker build -t user-service .
[+] Building 0.8s (9/9) FINISHED
 => [internal] load build definition from Dockerfile
 => => transferring dockerfile: 179B
 => [internal] load metadata for docker.io/library/node:18
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/4] FROM docker.io/library/node:18@sha256:c6ae79e38498325db67193d391e6ec1d224d96c693a8a4d943498556716d3783
 => [internal] load build context
 => => transferring context: 81.20kB
 => CACHED [2/4] WORKDIR /app
 => CACHED [3/4] COPY . /app
 => CACHED [4/4] RUN npm install
 => exporting to image
 => => exporting layers
 => => writing image sha256:cb128d4826d9c45a8819d0dcd1b7ed6d8579fb548a3ca33171a1ea4b8079b956
 => => naming to docker.io/library/user-service
[root@ip-172-31-120-88 user-service]#
```

```
[root@ip-172-31-120-88 user-service]# docker images
REPOSITORY                                                      TAG       IMAGE ID       CREATED              SIZE
user-service                                                    latest    cb128d4826d9   2 minutes ago        1.09GB
074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo   latest    573674bd266e   About an hour ago    1.09GB
<none>                                                          <none>    4947cbfecd55   About an hour ago    1.09GB
[root@ip-172-31-120-88 user-service]#
```

STEP 9: docker ps

```
[root@ip-172-31-120-88 user-service]# docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED              STATUS              PORTS                                       NAMES
659429fe3964   573674bd266e   "docker-entrypoint.s…"   About an hour ago   Up About an hour   0.0.0.0:3001->3000/tcp, :::3001->3000/tcp   cranky_n
cutt
5741c08149a6   573674bd266e   "docker-entrypoint.s…"   About an hour ago   Up About an hour   0.0.0.0:3000->3000/tcp, :::3000->3000/tcp   nifty_ja
[root@ip-172-31-120-88 user-service]#
```

STEP 10: Login Docker to ECR

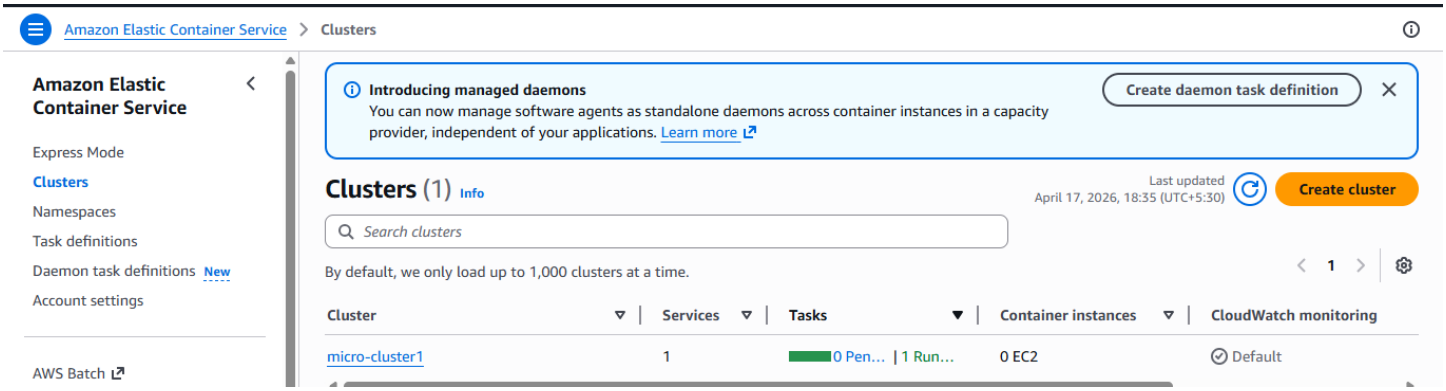
```
[ec2-user@ip-172-31-120-88 ~]$ aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-
stdin 123456789012.dkr.ecr.ap-south-1.amazonaws.com
WARNING! Your password will be stored unencrypted in /home/ec2-user/.docker/config.json.
Configure a credential helper to remove this warning. See
https://docs.docker.com/engine/reference/commandline/login/#credentials-store

Login Succeeded
```

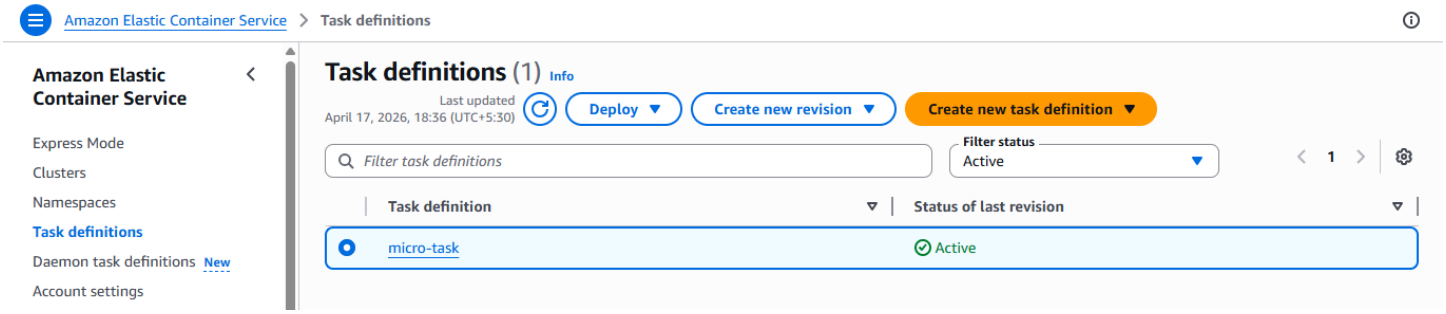
STEP 11: Tagged and pushed the Docker image

```
Login Succeeded
[ec2-user@ip-172-31-120-88 ~]$ ^C
[ec2-user@ip-172-31-120-88 ~]$ docker tag user-service:latest 074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo:latest
docker tag user-service:latest 074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo:latest
[ec2-user@ip-172-31-120-88 ~]$ docker push 074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo
Using default tag: latest
The push refers to repository [074139044318.dkr.ecr.ap-south-1.amazonaws.com/microservice-repo]
```

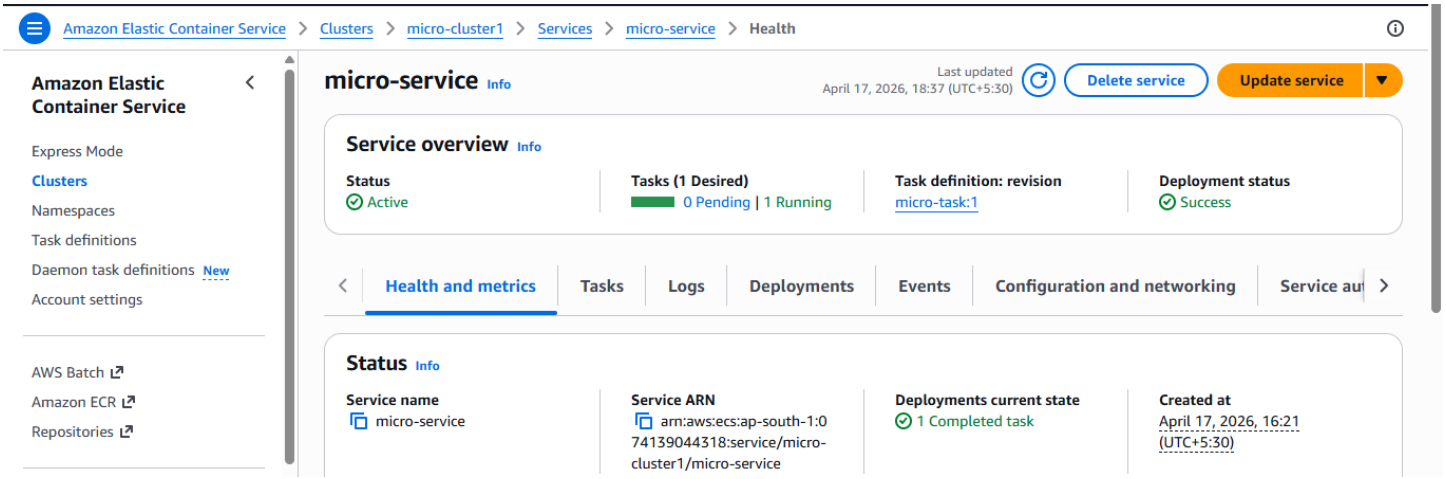
STEP 12: Create ECS Cluster



STEP 13: Create Task Definition



STEP 14: Create ECS Service

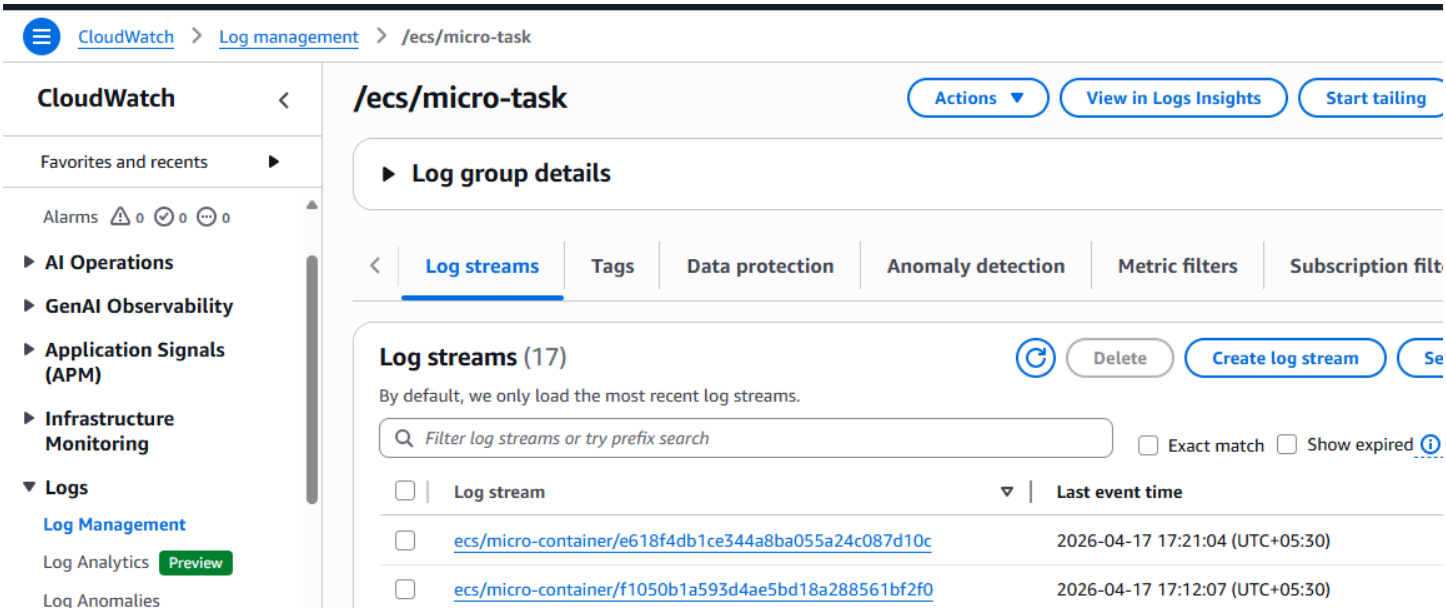


STEP 15: Access the Service

User Service Running 🚀

STEP 16: Monitoring Application Logs using AWS CloudWatch

- View Log Streams



- View Log Events

Troubleshooting:

- **Issue:** Docker images could not be pushed to Amazon ECR due to authentication errors.
- **Solution:** AWS CLI authentication was configured using the aws ecr get-login-password command to securely log in to the ECR repository before pushing the image.

Conclusion:

In conclusion, the Scalable Microservices Architecture using ECS and Fargate provides an easy and efficient way to deploy containerized applications. It allows services to run independently and scale based on demand without managing servers. With services like ECR for images, RDS for databases, and CloudWatch for monitoring, the system becomes more reliable and manageable. This approach improves scalability, performance, and simplifies overall application deployment in the cloud.